

HOME SOC SUITE

Implementation Priority Guide

Night's Watch — Build Roadmap & Weekly Work Plan

Reference: Use this document independently between sessions

HOW TO USE THIS DOCUMENT: This guide is designed to keep you productive between sessions. Each phase is self-contained — you can pick up any task, work on it independently, and resume from a clear stopping point. Tasks are ordered by impact and dependency. Complete Phase 1 fixes before building new collectors. The PowerShell patterns section gives you syntax references so you don't need to search from scratch.

PRIORITY OVERVIEW

Phase	Priority	What	Why First	Est. Sessions
Phase 1	CRITICAL	Fix existing collectors	Closes biggest gaps with minimal code	2-3
Phase 2	HIGH	Build Registry Warden + Harbinger	Persistence and execution monitoring	3-4
Phase 3	HIGH	Build Bloodhound DNS monitor	Closes major exfil channel	2
Phase 4	HIGH	Build Warden watchdog + FIM	Suite self-protection	2-3
Phase 5	MEDIUM	Build Security Event Log monitor	Account and privilege monitoring	2
Phase 6	MEDIUM	Build DoH Detector	Encrypted DNS evasion	1-2
Phase 7	MEDIUM	Python Correlation Engine	Cross-stream detection	5-8
Phase 8	MEDIUM	Situational modules	High-risk deployment protection	2-3
Phase 9	LOW	Python Dashboard	Visualization layer	5-8

PHASE 1 — FIX EXISTING COLLECTORS (Do This First)

These are single-line or small changes to scripts you already have. Each fix closes a confirmed critical gap.
Estimated time: 1-2 hours total.

Fix 1 — Sentinel.ps1: Remove PowerShell Exclusion [CRITICAL]

Find this line (approximately line 55):

```
if (-not $SeenConnections.ContainsKey($ID) -and $PName -ne "powershell") {
```

Replace with:

```
if (-not $SeenConnections.ContainsKey($ID)) {
```

Why: Why: PowerShell is the most common C2 channel for living-off-the-land attacks. Excluding it from logging is the single biggest blind spot in the suite.

Fix 2 — Sentinel.ps1: Add Time-Based Cache Expiry [HIGH]

Find the \$SeenConnections initialization at the top:

```
$SeenConnections = @{}
```

Replace with and add expiry check inside the loop:

```
$SeenConnections = @{}
$SeenConnectionTimes = @{}

# Inside the loop, before the connection check:
$ExpiredKeys = $SeenConnections.Keys | Where-Object {
  ((Get-Date) - $SeenConnectionTimes[$_]).TotalMinutes -gt 60
}
$ExpiredKeys | ForEach-Object {
  $SeenConnections.Remove($_)
  $SeenConnectionTimes.Remove($_)
}

# When adding to cache, also record the time:
$SeenConnections[$ID] = $true
$SeenConnectionTimes[$ID] = Get-Date
```

Why: Why: Currently a connection logged once is never logged again for the entire session. 60-minute expiry means persistent C2 connections get re-logged every hour.

Fix 3 — Steward.ps1: Log All Processes Above Threshold [HIGH]

Find the \$TopCPU line:

```
$TopCPU = $CPUResults | Sort-Object CPUPercent -Descending | Select-Object -First
$TopProcesses
```

Add a separate logging pass immediately after:

```
# Log ALL processes above threshold regardless of top 5 display
$CPUResults | Where-Object { $_.CPUPercent -ge $CPUUnknown } | ForEach-Object {
  $Severity = "OK"
  if ($_.CPUPercent -ge $CPUCritical) { $Severity = "CRITICAL" }
  elseif ($_.CPUPercent -ge $CPUSuspicious) { $Severity = "SUSPICIOUS" }
  elseif ($_.CPUPercent -ge $CPUUnknown) { $Severity = "UNKNOWN" }
  if ($Severity -ne "OK") {
    Write-Log $Severity "PROCESS: $(_.Name) at $(_.CPUPercent)% | RAM: $(_.RAM) MB"
  }
}
```

Why: Why: Currently only top 5 processes are logged. A malicious low-CPU process never in the top 5 is invisible to Ledger analysis.

Fix 4 — Sentinel.ps1 + Bulwark.ps1: ip-api Failure Handling [MEDIUM]

Find the geolocation failure catch block:

```
} catch { $IPCache[$R_IP] = 'Lookup Failed' }
```

Replace with:

```
} catch {  
$IPCache[$R_IP] = "Lookup Failed"  
# Log unknown IP with failed geolocation as SUSPICIOUS  
Write-Log "SUSPICIOUS" "GEOLOCATION FAILED: $R_IP - connection from $PName logged without location"  
}
```

Why: Why: Currently a failed geolocation silently continues with no alert. An attacker who blocks ip-api.com gets a free pass on location detection.

Fix 5 — Sentinel.ps1: Add Cumulative Transfer Tracking [HIGH]

Add at the top of the script after configuration:

```
$TransferTracker = @{}  
$TransferFile = "$RootPath\Config\Transfer_Baseline.json"  
if (Test-Path $TransferFile) {  
$TransferTracker = Get-Content $TransferFile | ConvertFrom-Json | ForEach-Object {  
$h = @{}; $_.PSObject.Properties | ForEach-Object { $h[$_.Name] = $_.Value }; $h  
}  
}
```

Then inside the connection loop, track and persist:

```
# Track cumulative transfer per process (approximate via connection count)  
$PKey = $PName.ToLower()  
if (-not $TransferTracker.ContainsKey($PKey)) { $TransferTracker[$PKey] = 0 }  
$TransferTracker[$PKey]++  
# Save every 10 cycles  
if ($SampleCount % 10 -eq 0) {  
$TransferTracker | ConvertTo-Json | Out-File $TransferFile -Encoding UTF8  
}
```

Why: Why: Without cross-session tracking an attacker can exfiltrate small amounts each boot session indefinitely without triggering cumulative thresholds.

PHASE 2A — BUILD REGISTRY WARDEN

New collector. Monitors registry run keys, SAM account RIDs, and service configuration keys for unauthorized changes. Polling interval: 60 seconds.

Script Structure

Registry_Warden.ps1 — skeleton to build from:

```
# Registry Warden – Skeleton
$RootPath = "$env:USERPROFILE\Desktop\SOC"
$LogFile = "$RootPath\Logs\RegistryWarden_Log.txt"
$BaselineFile = "$RootPath\Config\RegistryWarden_Baseline.json"
# KEYS TO MONITOR
$WatchKeys = @(
    "HKCU:\Software\Microsoft\Windows\CurrentVersion\Run",
    "HKLM:\Software\Microsoft\Windows\CurrentVersion\Run",
    "HKLM:\Software\Microsoft\Windows\CurrentVersion\RunOnce",
    "HKLM:\SYSTEM\CurrentControlSet\Services"
)
# Service config
function Get-RegistrySnapshot {
    $Snapshot = @{}
    foreach ($Key in $WatchKeys) {
        try {
            $Values = Get-ItemProperty -Path $Key -ErrorAction Stop
            $Snapshot[$Key] = $Values
        } catch {
            $Snapshot[$Key] = $null
        }
    }
    return $Snapshot
}
# SAM RID VALIDATION
function Test-AccountRIDs {
    $Users = Get-LocalUser
    foreach ($User in $Users) {
        # Guest should never have admin privileges
        if ($User.Name -eq "Guest" -and $User.Enabled) {
            Write-Log "CRITICAL" "RID CHECK: Guest account is enabled - verify RID assignment"
        }
    }
}
# MAIN LOOP: snapshot, compare, alert on differences
# Build diff logic comparing current snapshot to baseline
# Alert on any new value, changed value, or deleted value
```

Key implementation tasks:

- Capture baseline snapshot of all watched keys at startup
- Each cycle: take new snapshot and diff against baseline
- Alert on new key values (new run key added)
- Alert on changed key values (service binary path modified — RID hijack vector)
- Alert on deleted key values (covering tracks)
- Add SAM RID validation using Get-LocalUser — Guest should never be admin
- Save baseline to JSON, reload on restart to persist across sessions

PHASE 2B — BUILD HARBINGER

New collector. Logs all process creation events. Uses WMI event subscription for real-time detection rather than polling — this is the event-driven approach that closes the polling blind spot.

Script Structure

```
# Harbinger – Process Execution Logger – Skeleton
$RootPath = "$env:USERPROFILE\Desktop\SOC"
$LogFile = "$RootPath\Logs\Harbinger_Log.txt"
$BaselineFile = "$RootPath\Config\Harbinger_Baseline.json"
# HIGH RISK PATHS – alert immediately on execution from these
$HighRiskPaths = @(
    "$env:USERPROFILE\Downloads",
    "$env:TEMP",
    "$env:APPDATA",
    "C:\Users\Public"
)
# INTERPRETER BINARIES – alert if outside Program Files
$InterpreterNames = @(
    "python", "python3", "perl", "ruby", "lua", "node", "php"
)
# WMI EVENT SUBSCRIPTION – real-time process creation
$query = "SELECT * FROM Win32_ProcessStartTrace"
$action = {
    $Proc = $Event.SourceEventArgs.NewEvent
    $PName = $Proc.ProcessName
    $PID = $Proc.ProcessID
    # Get full path via Get-Process
    $PPath = (Get-Process -Id $PID -ErrorAction SilentlyContinue).Path
    # Check high risk path
    # Check interpreter outside Program Files
    # Log all new process creations
    # Compare against baseline
}
$watcher = Register-WmiEvent -Query $query -Action $action
# Keep script alive
while ($true) {
    Start-Sleep -Seconds 10
}
```

Key implementation tasks:

- Use Register-WmiEvent for real-time process creation events — not polling
- Log every process creation: name, PID, path, parent PID, parent name
- Alert CRITICAL on execution from high-risk paths (Downloads, Temp, AppData, Public)
- Alert HIGH on interpreter binaries (python/perl/ruby/lua) outside Program Files
- Build baseline of known-good processes after first week of normal use
- Alert UNKNOWN on any process not in baseline
- Track parent-child relationships — flag unusual parent processes

PHASE 3 — BUILD BLOODHOUND DNS MONITOR

New collector. Monitors DNS queries for exfiltration patterns, high-entropy domain names, and abnormal query volumes. Uses ETW (Event Tracing for Windows) or DNS Client event log.

```
# Bloodhound - DNS Monitor - Skeleton $RootPath = "$env:USERPROFILE\Desktop\SOC" $LogFile
= "$RootPath\Logs\Bloodhound_Log.txt" # DNS events are in
Microsoft-Windows-DNS-Client/Operational $DNSLog =
"Microsoft-Windows-DNS-Client/Operational" function Get-StringEntropy($String) { #
Calculate Shannon entropy of a string # High entropy (>3.5) suggests DGA or encoded data
$Chars = $String.ToCharArray() | Group-Object $Entropy = 0 foreach ($Char in $Chars) { $P
= $Char.Count / $String.Length $Entropy -= $P * [Math]::Log($P, 2) } return
[Math]::Round($Entropy, 2) } function Test-DGADomain($Domain) { $Parts =
$Domain.Split(".") $Subdomain = $Parts[0] $Entropy = Get-StringEntropy $Subdomain # Flag
if: length > 12 AND entropy > 3.5 return ($Subdomain.Length -gt 12 -and $Entropy -gt 3.5)
} # Query DNS event log for recent entries # Track query frequency per domain # Alert on
DGA-like domains # Alert on abnormal query volume # Alert on TXT record queries (DNS
exfiltration) # Alert on ICMP activity (separate from DNS but log here)
```

Key implementation tasks:

- Read from Microsoft-Windows-DNS-Client/Operational event log
- Implement Shannon entropy calculation for subdomain analysis
- Alert on high-entropy subdomains (length > 12, entropy > 3.5) — DGA indicator
- Alert on TXT record queries — classic DNS exfiltration channel
- Track query frequency per domain — alert on abnormal volume spikes
- Baseline normal DNS activity in first week
- Add ICMP monitoring via Get-NetAdapter statistics as separate section

PHASE 4 — BUILD WARDEN WATCHDOG + SCRIPT FIM

New collector. Monitors that all other collectors are running, verifies script file hashes against a signed manifest, and alerts if any script is modified without the maintenance flag being set.

```
# Warden - Watchdog + File Integrity Monitor - Skeleton $RootPath =
"$env:USERPROFILE\Desktop\SOC" $LogFile = "$RootPath\Logs\Warden_Log.txt" $ManifestFile =
"$RootPath\Config\Warden_Manifest.json" $MaintenanceFlag =
"$RootPath\Config\maintenance.flag" # SCRIPTS TO MONITOR $MonitoredScripts = @(
"$RootPath\Collectors\Sentinel.ps1", "$RootPath\Collectors\Bulwark.ps1",
"$RootPath\Collectors\Steward.ps1", "$RootPath\Collectors\CITYGUARD.ps1",
"$RootPath\Collectors\Watchman.ps1", "$RootPath\Collectors\Registry_Warden.ps1",
"$RootPath\Collectors\Harbinger.ps1", "$RootPath\Collectors\Bloodhound.ps1" ) function
Get-FileHashSHA256($Path) { return (Get-FileHash -Path $Path -Algorithm SHA256).Hash }
function Build-Manifest { # Run this ONCE after each legitimate update # Creates the
known-good hash manifest $Manifest = @{} foreach ($Script in $MonitoredScripts) { if
(Test-Path $Script) { $Manifest[$Script] = Get-FileHashSHA256 $Script } } $Manifest |
ConvertTo-Json | Out-File $ManifestFile -Encoding UTF8 Write-Host "[+] Manifest built:
$(($Manifest.Count) scripts hashed" -ForegroundColor Green } function Test-Integrity { if
(-not (Test-Path $ManifestFile)) { Write-Log "CRITICAL" "MANIFEST MISSING - run
Build-Manifest after setup" return } if (Test-Path $MaintenanceFlag) { Write-Log "INFO"
"Maintenance flag set - skipping integrity check" return } $Manifest = Get-Content
$ManifestFile | ConvertFrom-Json foreach ($Script in $MonitoredScripts) { $Expected =
$Manifest.$Script $Actual = Get-FileHashSHA256 $Script if ($Expected -ne $Actual) {
Write-Log "CRITICAL" "INTEGRITY VIOLATION: $Script hash mismatch" # Optional:
self-terminate other collectors here } } } # COLLECTOR HEALTH CHECK function
Test-CollectorHealth { # Check each collector script has a running PowerShell process #
This is approximate - match by window title or scheduled task $RunningScripts =
Get-Process powershell,pwsh -ErrorAction SilentlyContinue | ForEach-Object {
$_.MainWindowTitle } # Alert if expected collectors are not running } # MAIN LOOP while
($true) { Test-Integrity Test-CollectorHealth Start-Sleep -Seconds 300 # Check every 5
minutes }
```

Key implementation tasks:

- Build-Manifest function — run once after each legitimate update, creates SHA256 hash of all scripts
- Maintenance flag — a file that tells Warden a legitimate update is in progress
- Test-Integrity — runs every 5 minutes, compares current hashes to manifest
- Alert CRITICAL if any script hash differs without maintenance flag
- Test-CollectorHealth — verify expected collector processes are running
- Alert if any collector stops unexpectedly
- Protect Warden itself — Warden's own hash should be checked by a scheduled task

PHASE 5 — BUILD SECURITY EVENT LOG MONITOR

New collector. Monitors Windows Security Event Log for logon anomalies, account creation, privilege escalation, and after-hours access. Requires Administrator elevation.

```
# Security Event Log Monitor – Skeleton $RootPath = "$env:USERPROFILE\Desktop\SOC"
$LogFile = "$RootPath\Logs\SecEventLog_Log.txt" # EVENT IDs TO MONITOR $WatchEventIDs = @(
4624, # Successful logon 4625, # Failed logon 4648, # Logon with explicit credentials
4720, # User account created 4722, # User account enabled 4728, # Member added to security
group 4732, # Member added to local admin group 4756, # Member added to universal security
group 4776, # Credential validation 4798, # User local group membership enumerated 4799 #
Security-enabled local group membership enumerated ) function Get-SecurityEvents($Minutes
= 10) { $Since = (Get-Date).AddMinutes(-$Minutes) Get-WinEvent -LogName Security
-ErrorAction SilentlyContinue | Where-Object { $_.Id -in $WatchEventIDs -and
$_ .TimeCreated -gt $Since } } function Get-EventSeverity($EventID, $Hour) { switch
($EventID) { 4720 { return "CRITICAL" } # Account created 4728 { return "CRITICAL" } #
Added to security group 4732 { return "CRITICAL" } # Added to admin group 4625 { if ($Hour
-ge 0 -and $Hour -lt 6) { return "SUSPICIOUS" } return "UNKNOWN" } 4624 { if ($Hour -ge 0
-and $Hour -lt 6) { return "SUSPICIOUS" } return "OK" } default { return "UNKNOWN" } } } #
MAIN LOOP # Poll every 60 seconds # Track LastEventTime to avoid duplicate logging # Alert
on account creation immediately # Alert on after-hours logons # Track failed logon count –
alert on brute force pattern
```

PHASE 6 — BUILD DOH DETECTOR

New collector. Detects DNS over HTTPS evasion by monitoring HTTPS connections to known DoH resolver IPs from non-browser processes.

```
# DoH Detector – Skeleton $RootPath = "$env:USERPROFILE\Desktop\SOC" $LogFile =
"$RootPath\Logs\DoHDetector_Log.txt" # KNOWN DOH RESOLVER IPs $DoHResolvers = @(
"1.1.1.1", # Cloudflare "1.0.0.1", # Cloudflare secondary "8.8.8.8", # Google "8.8.4.4", #
Google secondary "9.9.9.9", # Quad9 "149.112.112.112", # Quad9 secondary "208.67.222.222",
# OpenDNS "176.103.130.130" # AdGuard ) # KNOWN BROWSER PROCESSES – these are allowed to
use DoH $AllowedDoHProcesses = @( "chrome","firefox","msedge","brave","opera","vivaldi" )
while ($true) { $Conns = Get-NetTCPConnection -State Established -ErrorAction
SilentlyContinue | Where-Object { $_.RemoteAddress -in $DoHResolvers -and $_.RemotePort
-eq 443 } foreach ($C in $Conns) { $Proc = Get-Process -Id $C.OwningProcess -ErrorAction
SilentlyContinue $PName = if ($Proc) { $Proc.Name.ToLower() } else { "unknown" } if
($PName -notin $AllowedDoHProcesses) { Write-Log "CRITICAL" "DOH EVASION: $PName
connecting to DoH resolver $($C.RemoteAddress)" } } Start-Sleep -Seconds 5 }
```


PHASE 7 — PYTHON CORRELATION ENGINE (Overview)

Build after all collectors are complete. The engine reads log files from all collectors, applies time-window correlation rules, and outputs a structured alert log. Build in Python.

Architecture:

- `log_parser.py` — reads and parses all collector log files into structured events
- `correlator.py` — applies the 26 correlation rules with configurable time windows
- `alert_manager.py` — deduplicates, throttles (Rule 18), and writes alert log
- `engine.py` — main loop, polls every 5-10 minutes, orchestrates all modules
- `config.py` — all thresholds and time windows in one place (rotate periodically)

Start with these 3 rules first (highest value, simplest to implement):

- Rule 6 — LOLBin Network Activity: Sentinel log + known LOLBin list = immediate CRITICAL
- Rule 13 — Initial Access: Harbinger log (Downloads/Temp path) + Sentinel log (outbound within 120s)
- Rule 3 — Persistence + C2: CityGuard log (new task) + Sentinel log (new connection within 10 min)

PHASE 8 — SITUATIONAL MODULES (Optional Deployment)

These three modules are activated only for high-risk deployment scenarios. They do not run as permanent background collectors.

HID_Monitor.ps1 — Rule 23

Monitor new HID device registration via WMI `Win32_DeviceChangeEvent`. Alert on new keyboard-class device. Optionally lock workstation or kill powershell/cmd on trigger.

Implementation approach: `Register-WmiEvent -Query 'SELECT * FROM Win32_DeviceChangeEvent'` — filter for keyboard class devices

WiFi_Ghost.ps1 — Rule 24

Baseline BSSID on first connection to each known SSID using `netsh`. Each connection check whether BSSID matches baseline. Alert and optionally disconnect on mismatch.

Implementation approach: `netsh wlan show interfaces | Select-String BSSID` — parse MAC address, compare to baseline

Focus_Thief.ps1 — Rule 25

Monitor foreground focus changes using Win32 API `P/Invoke`. Alert when focus changes in locked/idle state or when hidden window is created.

Implementation approach: `Add-Type -TypeDefinition` with `P/Invoke` to `user32.dll` `GetForegroundWindow()` and `GetWindowThreadProcessId()`

Visit_Integrity.ps1 — Rule 26

Pre-visit: snapshot all running processes, scheduled tasks, listening ports, registry run keys, recent DNS queries. Post-visit: diff against snapshot and generate report of everything that changed during the visit window.

Implementation approach: Run before leaving for client visit. Run again on return. Compare outputs. Any diff during visit window is a candidate for investigation.

POWERSHELL QUICK REFERENCE

Common patterns used across the suite. Reference these when building new collectors.

Pattern	Code
File hashing	<code>Get-FileHash -Path 'file.ps1' -Algorithm SHA256</code>
WMI event subscription	<code>Register-WmiEvent -Query 'SELECT * FROM Win32_ProcessStartTrace' -Action { \$Event.SourceEvent</code>
Read event log	<code>Get-WinEvent -LogName Security Where-Object { \$_.Id -in @(4624,4720) -and \$_.TimeCreated -</code>
Get process path	<code>(Get-Process -Id \$PID -ErrorAction SilentlyContinue).Path</code>
Get TCP connections	<code>Get-NetTCPConnection -State Established Where-Object { \$_.RemoteAddress -notlike '127.*' }</code>
Registry read	<code>Get-ItemProperty -Path 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run'</code>
Get local users	<code>Get-LocalUser Select-Object Name, Enabled, PasswordRequired</code>
Network interface BSSID	<code>netsh wlan show interfaces Select-String 'BSSID'</code>
HID devices	<code>Get-PnpDevice Where-Object { \$_.Class -eq 'Keyboard' }</code>
P/Invoke user32	<code>Add-Type -TypeDefinition 'using System.Runtime.InteropServices; public class Win32 { [DllImp</code>
Write structured log	<code>"[\$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')] [SEV] Msg" Out-File \$LogFile -Append -Encodin</code>
Convert to/from JSON	<code>\$Object ConvertTo-Json -Depth 5 Out-File \$File; Get-Content \$File ConvertFrom-Json</code>
Shannon entropy function	<code>function Get-Entropy(\$S) { (\$S.ToCharArray() Group-Object ForEach-Object { \$p=\$_.Count/\$</code>
Compare registry snapshots	<code>Compare-Object (\$Baseline.Keys) (\$Current.Keys) # then compare values for matching keys</code>
Check scheduled tasks	<code>Get-ScheduledTask Select-Object TaskName, TaskPath, State</code>

WEEKLY WORK PLAN — KEEP YOURSELF BUSY

Use this as a daily task list when you have time between Umschulung sessions. Each task is self-contained — you can stop and restart without losing progress.

Day/Session	Task	Output	Difficulty
Session 1	Apply Fix 1 & 2 to Sentinel.ps1 (PowerShell exclusion + update check fix)	Updated Sentinel.ps1	Easy
Session 2	Apply Fix 3 to Steward.ps1 (log all above threshold) + Fix 4 (apply Stewarding)	Updated Stewarding.ps1	Easy
Session 3	Apply Fix 5 to Sentinel.ps1 (cumulative transfer tracking)	Updated Sentinel.ps1	Medium
Session 4	Build Registry_Warden.ps1 skeleton — baseline capture + new collector draft	New collector draft	Medium
Session 5	Complete Registry_Warden.ps1 — add SAM RID check + Registry_Warden.ps1 tooling	Registry_Warden.ps1 tooling	Medium
Session 6	Build Harbinger.ps1 skeleton — WMI event subscription + new collector creation	New collector creation	Medium-Hard
Session 7	Complete Harbinger.ps1 — high risk path detection + integrity checks down	Harbinger.ps1 v1	Medium
Session 8	Build Bloodhound.ps1 — DNS event log reader + entropy new collector draft	New collector draft	Medium
Session 9	Complete Bloodhound.ps1 — DGA detection + TXT queuing	Bloodhound.ps1 v1	Medium
Session 10	Build Warden.ps1 — Build-Manifest function + integrity checks	Warden.ps1 v1	Medium
Session 11	Build DoH_Detector.ps1 — full implementation (small scope)	DoH_Detector.ps1 v1	Easy
Session 12	Build Security_EventLog.ps1 — event ID monitoring + a Security.ps1 v1	Security.ps1 v1	Easy-Medium
Session 13	Build Visit_Integrity.ps1 — pre/post visit snapshot and diff	Visit_Integrity.ps1 v1	Medium
Session 14	Build WiFi_Ghost.ps1 + HID_Monitor.ps1 (situational modules)	Few situational scripts	Medium
Session 15+	Python correlation engine — start with log_parser.py	Engine foundation	Hard — do with Claude

IMPORTANT NOTE ON SESSION 15+: The Python correlation engine is the most complex component. Don't attempt it solo until all PowerShell collectors are complete and tested. When you're ready, start a fresh session with Claude and bring this document. The architecture is already defined — the build session will be focused and efficient.